CASE STUDY 3

Unit III: Stacks, Queues and Linked Lists

Implementation of Stack and Queue Operations Using Linked Lists

Aim

To implement stack and queue operations using linked lists.

Objectives

- Understand LIFO and FIFO.
- Implement push and pop for a stack.
- Implement enqueue and dequeue for a queue.

Theory

A linked list contains nodes, and each node stores data and a link to the next node. A stack follows LIFO (Last In, First Out), so the latest inserted item is removed first. A queue follows FIFO (First In, First Out), so the earliest inserted item is removed first. A stack uses push and pop; a queue uses enqueue and dequeue.

Algorithm

1. Stack push: create a node and link it before the current top.
2. Stack pop: remove and return the top node.
3. Queue enqueue: add a node at the rear.
4. Queue dequeue: remove and return the front node.

Python Program

```
# Stack and Queue using linked lists
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        node = Node(data)
        node.next = self.top
        self.top = node

    def pop(self):
        if self.top is None:
            return "Stack is empty"
        data = self.top.data
        self.top = self.top.next
        return data

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        node = Node(data)
        if self.rear is None:
            self.front = self.rear = node
        else:
            self.rear.next = node
            self.rear = node

    def dequeue(self):
        if self.front is None:
            return "Queue is empty"
        data = self.front.data
```

```
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        return data

s = Stack()
s.push(10)
s.push(20)
print("Stack pop:", s.pop())

q = Queue()
q.enqueue(10)
q.enqueue(20)
print("Queue dequeue:", q.dequeue())
```

Sample Output

```
Stack pop: 20\nQueue dequeue: 10
```

Conclusion

Linked lists allow stack and queue operations to be performed dynamically without a fixed-size array.